Solution Requirements

Date	30 September 2026
Team ID	SWTID-2026-2032
Project Name	Learnmate – AI StudyBuddy
Maximum Marks	4 Marks

Define Functional and Non-Functional Requirements:

This is a list of requirements that define exactly what Learnmate will do and how well it will perform.

- **Functional Requirements (FR)** define specific behaviors, functions, or features of the system (what the system should *do*).
- **Non-Functional Requirements (NFR)** specify the criteria that judge the operation of a system, rather than specific behaviors (how the system should *be*).

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority (High/ Medium/Low)
1	Authentication	Users register with name, email and password (passwords hashed with bcrypt) and log in to receive a signed JWT access token. Every protected request must carry the token as a Bearer header; missing, invalid or expired tokens are rejected with 401.	High
2	Authorization levels	Two roles: **student** (default) can upload materials and use AI-generated content; **admin** can additionally access admin routes to manage users and materials. Non-admins receive 403 on admin routes.	High
3	External interfaces	Google Gemini API (via @google/generative-ai) for generating summaries, flashcards, quizzes and study plans; MongoDB (via Mongoose) for storing users and materials; local server storage for uploaded files.	High
4	Transactions processing	No payments. Core transaction flow: upload file (.txt, .md, .pdf, max 5 MB) → extract text → send prompt to Gemini → save summary, flashcards, quiz and study plan against the user's material record. Temporary AI errors (429/503) are retried automatically.	High
5	Reporting	Students can view their saved materials with summary, flashcards, quiz and study plan. Admins can view lists of users and uploaded materials.	Medium
6	Business rules, etc.	Only .txt, .md and .pdf files are accepted; files above 5 MB are rejected. Each material belongs to one user. Email must be unique and is stored in lowercase. New accounts default to the student role.	High
7	Compliance to laws or regulations	Passwords are never stored in plain text; secrets (JWT secret, API keys, database URI) are kept in environment variables. Uploaded study content is treated as private user data and must only be accessible to its owner and admins.	Medium

8	Other	Health-check endpoint (GET /) to confirm the API is running; global error handler returning JSON messages; CORS restricted to the configured client URL.	Low

Step 2: Non-Functional Requirements (NFR)

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
1	Performance & Speed	Regular API calls (login, fetching materials) respond quickly; AI generation depends on Gemini response time and is handled asynchronously with clear waiting feedback.	Non-AI API responses in under 2 seconds; AI generation completes in under 60 seconds
2	Scalability	Stateless JWT authentication and a MongoDB database allow the API to scale horizontally as users and materials grow.	Supports at least 500 concurrent users (target)
3	Security & Data Privacy	Passwords hashed with bcrypt (10 salt rounds); JWT verification on protected routes; role check for admin routes; file-type and size validation on uploads; secrets kept in .env.	No plain-text passwords; 100% of protected routes require a valid token; HTTPS in production
4	Reliability & Availability	Gemini calls are retried up to 3 times with exponential backoff (5 s, 10 s, 20 s) on 429/503 errors; errors are returned as clear JSON messages instead of crashing the server.	99% uptime per month (target); failed AI calls recover automatically in most temporary outages
5	Usability & Accessibility	Simple sign-up, upload and study screens; clear error messages for rejected files or failed requests; layout works on desktop and mobile.	A new user can upload a file and get results in under 3 minutes; responsive on common screen sizes
6	Other	Maintainability: code organized in controllers, routes, models, middleware and utils. Portability: runs on any Node.js host with MongoDB.	Modular structure kept; setup completed with npm install and a .env file